

AIML LAB

1. A) Simple linear regression model for headBrain

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Reading Data
data = pd.read_csv('headbrain.csv')
print(data.shape)
data.head()

# Collecting X and Y
X = data['Head Size(cm^3)'].values
Y = data['Brain Weight(grams)'].values

# Calculating coefficient

# Mean X and Y
mean_x = np.mean(X)
mean_y = np.mean(Y)

print(mean_x)
print(mean_y)

# Total number of values
n = len(X)
print(n)

# Using the formula to calculate b1 and b0
numer = 0
denom = 0
for i in range(n):
    numer += (X[i] - mean_x) * (Y[i] - mean_y)
    denom += (X[i] - mean_x) ** 2

b1 = numer / denom
b0 = mean_y - (b1 * mean_x)

# Printing coefficients
print("Coefficients")
print(b1, b0)

# Plotting Values and Regression Line

max_x = np.max(X) + 100
min_x = np.min(X) - 100

# Calculating line values x and y
x = np.linspace(min_x, max_x, 1000)
y = b0 + b1 * x

# Plotting Line
plt.plot(x, y, color='#58b970', label='Regression Line')

# Plotting Scatter Points
plt.scatter(X, Y, c='#ef5423', label='Scatter Plot')
```

```
plt.xlabel('Head Size in cm3')
plt.ylabel('Brain Weight in grams')
plt.legend()
plt.show()
```

```
# Calculating R2 Score
ss_tot = 0
ss_res = 0
for i in range(n):
    y_pred = b0 + b1 * X[i]
    ss_tot += (Y[i] - mean_y) ** 2
    ss_res += (Y[i] - y_pred) ** 2

r2 = 1 - (ss_res / ss_tot)
print("R2 Score")
print(r2)
```

OUTPUT :--

R2 Score

0.6393117199570003 The model shows a moderate fit.

B) Simple linear regression model for housing_prices_SLR dataset

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Load dataset
df = pd.read_csv("housing_prices_SLR.csv")
print(df.head())
# Feature and Target
x = df[['AREA']].values
y = df['PRICE'].values

# Split the data (80% train, 20% test)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=100)

# Train the model
model = LinearRegression()
model.fit(x_train, y_train)

# Predict
y_pred_train = model.predict(x_train)
y_pred_test = model.predict(x_test)

# (i) Analyze R2 score
print("R2 Score (Training):", r2_score(y_train, y_pred_train))
print("R2 Score (Testing):", r2_score(y_test, y_pred_test))

# (ii) Display data points with fitted model
plt.scatter(df['AREA'], df['PRICE'], color='blue', label='Actual Data')
plt.plot(df['AREA'], model.predict(df[['AREA']].values), color='red', label='Fitted Model')
plt.xlabel("Area (sq.ft)")
plt.ylabel("Price")
plt.title("Housing Prices vs Area (Linear Regression Fit)")
```

```
plt.legend()
plt.show()
```

Output :----

R² Score (Training): 0.865632112064227

R² Score (Testing): 0.570954870212099

The model gives a good fit for training data and a moderate fit for testing data

2. a) Multiple linear regression model for student dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Step 1: Load dataset
data = pd.read_excel('student.xlsx') # ensure file name matches exactly
print(data.head())
math = data['Math'].values
read = data['Reading'].values
write = data['Writing'].values

# Step 2: Scatter Plot (Visualization)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(math, read, write, color='red')
plt.title("Math vs Reading vs Writing")
plt.show() # ✓ must be uncommented to render + continue

# Step 3: Prepare Data
m = len(math)
X = np.c_[np.ones(m), math, read] # Add intercept term
Y = write
B = np.zeros(3)
alpha = 0.0001 # learning rate

# Step 4: Gradient Descent
def gradient_descent(X, Y, B, alpha, iters):
    for _ in range(iters):
        B -= alpha * (X.T @ (X @ B - Y)) / len(Y)
    return B

# Step 5: Train Model
B = gradient_descent(X, Y, B, alpha, 100000)
print("\nCoefficients:", B)

# Step 6: Predictions
Y_pred = X @ B

# Step 7: R2 Score
r2 = 1 - np.sum((Y - Y_pred)**2) / np.sum((Y - np.mean(Y))**2)
print("R2 Score:", round(r2, 4))
```

OUTPUT:--

Coefficients: [-0.47889172 0.09137252 0.90144884]

R² Score: 0.9097

The model's R² score of 0.9097 indicates an excellent (very strong) fit

2 b) Multiple linear regression model for housing_prices dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

df = pd.read_excel("housing_prices.xlsx")
df
df = df.iloc[:, [0,1,2,4]]
df.head()

x = df.iloc[:,3].values
y = df.iloc[:,3].values
print(x[:5])
print(y[:5])

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=100)

print(x_train.shape)
print(x_test.shape)
print(x_train.shape)
print(x_test.shape)

from sklearn.linear_model import LinearRegression
mlr_model = LinearRegression(fit_intercept=True)
mlr_model.fit(x_train, y_train)

print(mlr_model.intercept_) # (PRICE=(n)+n)*AREA
print(mlr_model.coef_)     # y = c + mx

# Added R2 score labels
print("R2 Score (Training Data):", mlr_model.score(x_train, y_train))
print("R2 Score (Testing Data):", mlr_model.score(x_test, y_test))
```

Output :--- R² Score (Training Data): 0.9646204930750824
R² Score (Testing Data): 0.8606154687481633

The model's R² scores of 0.9646 (training) and 0.8606 (testing) indicate an excellent fit

3) Apply:

Decision tree and Naïve Bayesian classifiers on breast cancer dataset. Find out

i) No of benign and malignant cases in the testing phase

ii) Predict the accuracy of the both classifiers

#3a Decision Tree Classifier on Breast Cancer Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

##### Step 2 : Load the csv/excel file into pandas dataframe and clean the data
df = pd.read_excel('breast_cancer.xlsx')
df
df = df.iloc[:, :-1]
df
df.head()
```

```

x = df.iloc[:, 2:].values
print(x)
y = df.diagnosis.values
#print(y)
print(x[:2])
print(y[:5])

#### Step 4 : Split the data into training set and test set
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2)

#### Step 5 : Instantiate a decision tree model and train the model
from sklearn.tree import DecisionTreeClassifier
dt_classifier = DecisionTreeClassifier()
dt_classifier.fit(x_train, y_train)

#### Step 6 : Use the model to predict the class labels for new data
predictions = dt_classifier.predict(x_test)
prob_predictions = dt_classifier.predict_proba(x_test)
print(predictions)
print(prob_predictions)

#### Step 7 : Calculate Accuracy score and confusion matrix for train and test data
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
print("Training accuracy Score is : ", accuracy_score(y_train, dt_classifier.predict(x_train)))
print("Testing accuracy Score is : ", accuracy_score(y_test, dt_classifier.predict(x_test)))
print("Training Confusion Matrix is : \n", confusion_matrix(y_train,
dt_classifier.predict(x_train)))
print("Testing Confusion Matrix is : \n", confusion_matrix(y_test,
dt_classifier.predict(x_test)))
print(classification_report(y_test, dt_classifier.predict(x_test)))

```

Output:---Training accuracy Score is : 1.0
Testing accuracy Score is : 0.9122807017543859
Training Confusion Matrix is :
[[286 0]
[0 169]]
Testing Confusion Matrix is :
[[64 7]
[3 40]]

	precision	recall	f1-score	support
B	0.96	0.90	0.93	71
M	0.85	0.93	0.89	43
accuracy			0.91	114
macro avg	0.90	0.92	0.91	114
weighted avg	0.92	0.91	0.91	114

3b) # Naive Bayesian Classification ,

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.naive_bayes import GaussianNB

# Step1: Read data as dataframe
df = pd.read_excel('breast_cancer.xlsx')
df = df.iloc[:, :-1]

```

```

x = df.iloc[:, 2:].values
y = df.diagnosis.values

# Split data
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=500)

# Baseline model
baseline_pred = ["B"] * len(y_train)
print("Baseline Accuracy:", accuracy_score(y_train, baseline_pred))
print(confusion_matrix(y_train, baseline_pred))
print(classification_report(y_train, baseline_pred))

# Naive Bayes Model
nb_model = GaussianNB()
nb_model.fit(x_train, y_train)

print("Training Accuracy:", nb_model.score(x_train, y_train))
print("Testing Accuracy:", nb_model.score(x_test, y_test))

print("Training Confusion Matrix:\n", confusion_matrix(y_train,
nb_model.predict(x_train)))
print("Testing Confusion Matrix:\n", confusion_matrix(y_test,
nb_model.predict(x_test)))

print("Training Classification Report:\n", classification_report(y_train,
nb_model.predict(x_train)))
print("Testing Classification Report:\n", classification_report(y_test,
nb_model.predict(x_test)))

# Threshold adjustment
pred = nb_model.predict_proba(x_train)[:, 1] > 0.25
print("Training Confusion (Threshold=0.25):\n", confusion_matrix(y_train == 'M',
pred))

test_pred = nb_model.predict_proba(x_test)[:, 1] > 0.25
print("Testing Confusion (Threshold=0.25):\n", confusion_matrix(y_test == 'M',
test_pred))

```

Output:--

Baseline Accuracy: 0.610989010989011

```
[[278  0]
 [177  0]]
```

	precision	recall	f1-score	support
B	0.61	1.00	0.76	278
M	0.00	0.00	0.00	177
accuracy			0.61	455
macro avg	0.31	0.50	0.38	455
weighted avg	0.37	0.61	0.46	455

Training Accuracy: 0.9340659340659341

Testing Accuracy: 0.9736842105263158

Training Confusion Matrix:

```
[[270  8]
 [ 22 155]]
```

Testing Confusion Matrix:

```
[[78  1]
 [ 2 33]]
```

Training Classification Report:

	precision	recall	f1-score	support
B	0.92	0.97	0.95	278
M	0.95	0.88	0.91	177
accuracy			0.93	455
macro avg	0.94	0.92	0.93	455
weighted avg	0.93	0.93	0.93	455

Testing Classification Report:

	precision	recall	f1-score	support
B	0.97	0.99	0.98	79
M	0.97	0.94	0.96	35
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

Training Confusion (Threshold=0.25):

```
[[269  9]
 [ 19 158]]
```

Testing Confusion (Threshold=0.25):

```
[[78  1]
 [ 2 33]]
```